

MANUAL DE INSTALACIÓN, DESPLIEGUE Y SEGURIDAD

Gestión de la Infraestructura de RosterApp

Autor: Ignacio Leo Sánchez

Centro: IES Albarregas - 2º DAW

1. Preparación del Entorno Local

Para garantizar la correcta ejecución del código fuente y evitar problemas de compatibilidad, la máquina de desarrollo debe cumplir con las siguientes herramientas instaladas:

- **Node.js:** Versión 18.x LTS o superior.
- **Gestor de Paquetes:** npm (incluido con Node.js) o bun.
- **Control de Versiones:** Git instalado y configurado.

2. Configuración e Instalación del Proyecto

Una vez clonado el repositorio en el equipo local, es estrictamente necesario configurar las credenciales de comunicación con el servicio BaaS antes de lanzar la aplicación. Siga los pasos detallados a continuación:

2.1. Variables de Entorno

Cree un archivo denominado `.env` en el directorio raíz del proyecto. Este archivo contiene secretos críticos y está excluido del control de versiones mediante el `.gitignore`. Inserte las siguientes claves obtenidas desde el panel de control de su proyecto en Supabase:

```
VITE_SUPABASE_URL=https://[ID_DEL_PROYECTO].supabase.co  
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

2.2. Inicialización con Vite

Con las variables configuradas, proceda a descargar las dependencias listadas en el `package.json` y a levantar el servidor de desarrollo:

```
# 1. Instalación de paquetes (React, Tailwind v4, React Router, Supabase JS)
```

```
npm install
```

```
# 2. Arranque del servidor de desarrollo local  
npm run dev
```

3. Configuración de Base de Datos y Seguridad (Supabase)

El proyecto delega la gestión de la base de datos PostgreSQL a Supabase. Para garantizar la confidencialidad de los datos (por ejemplo, que un empleado no pueda acceder a los fichajes o nóminas de otro), se ha implementado **Row Level Security (RLS)**.

- **Autenticación:** El registro de nuevos usuarios en Supabase Auth debe configurarse para aceptar registros de correo electrónico.
- **Políticas RLS:** En la tabla de fichajes, se debe establecer una política restrictiva: `(auth.uid() = empleado_id)`. Esto asegura a nivel de base de datos que, incluso si alguien intercepta el token JWT, la API solo devolverá las filas que pertenezcan al identificador único del usuario logueado.
- **Roles de Administrador:** Se recomienda crear una tabla anexa roles o utilizar los *claims* del JWT de Supabase para validar si el `auth.uid()` corresponde a un administrador, otorgándole permisos de lectura/escritura globales.

4. Despliegue en Producción (Vercel)

El pase a producción de RosterApp se realiza mediante un flujo de despliegue continuo en Vercel. Este servicio compila el proyecto ejecutando `npm run build` y sirve el directorio `dist/` resultante en una red de distribución de contenido (CDN) global.

4.1. Prevención de Errores 404 (Serverless Routing)

Al ser una Single Page Application con React Router, la navegación interna modifica la URL en el navegador sin realizar peticiones físicas al servidor. Si un usuario intenta acceder directamente a una sub-ruta (ej. `/admin/informes`), el servidor Vercel no encontrará esa carpeta y arrojará un error 404. Para solucionarlo, el proyecto incluye un archivo `vercel.json` que redirige el tráfico al `index` de la aplicación.

```
{  
  "rewrites": [  
    {  
      "source": "/(.*)",  
      "destination": "/index.html"  
    }  
  ]  
}
```

```
}  
]  
}
```

4.2. Pasos Finales en el Dashboard de Vercel

1. Vincule su repositorio de GitHub/GitLab con Vercel.
2. En el paso de configuración del proyecto, navegue a **Environment Variables**.
3. Inyecte las mismas variables `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY` de su entorno local.
4. Haga clic en **Deploy**. Tras la compilación, Vercel le proporcionará la URL pública del proyecto.